

CSEg 1101 Introduction to Computing

Lecture 1



- Goals of the course
- About Python
- What is computation and Computational thinking?
- 2D robot control

Two-level goals

- Building up a basis on ICT (Information and Communications Technology)
- This course is about programming and computational thinking, not about learning a programming language.

Think like a computer scientist for problem solving !

- Python is a programming language that is easy to learn and very powerful.
- Main language used for web programming at Google.
- Python is a programming language that is easy to learn and very powerful.
- Widely used in scientific computation, for instance at NASA, by mathematicians and physicists.

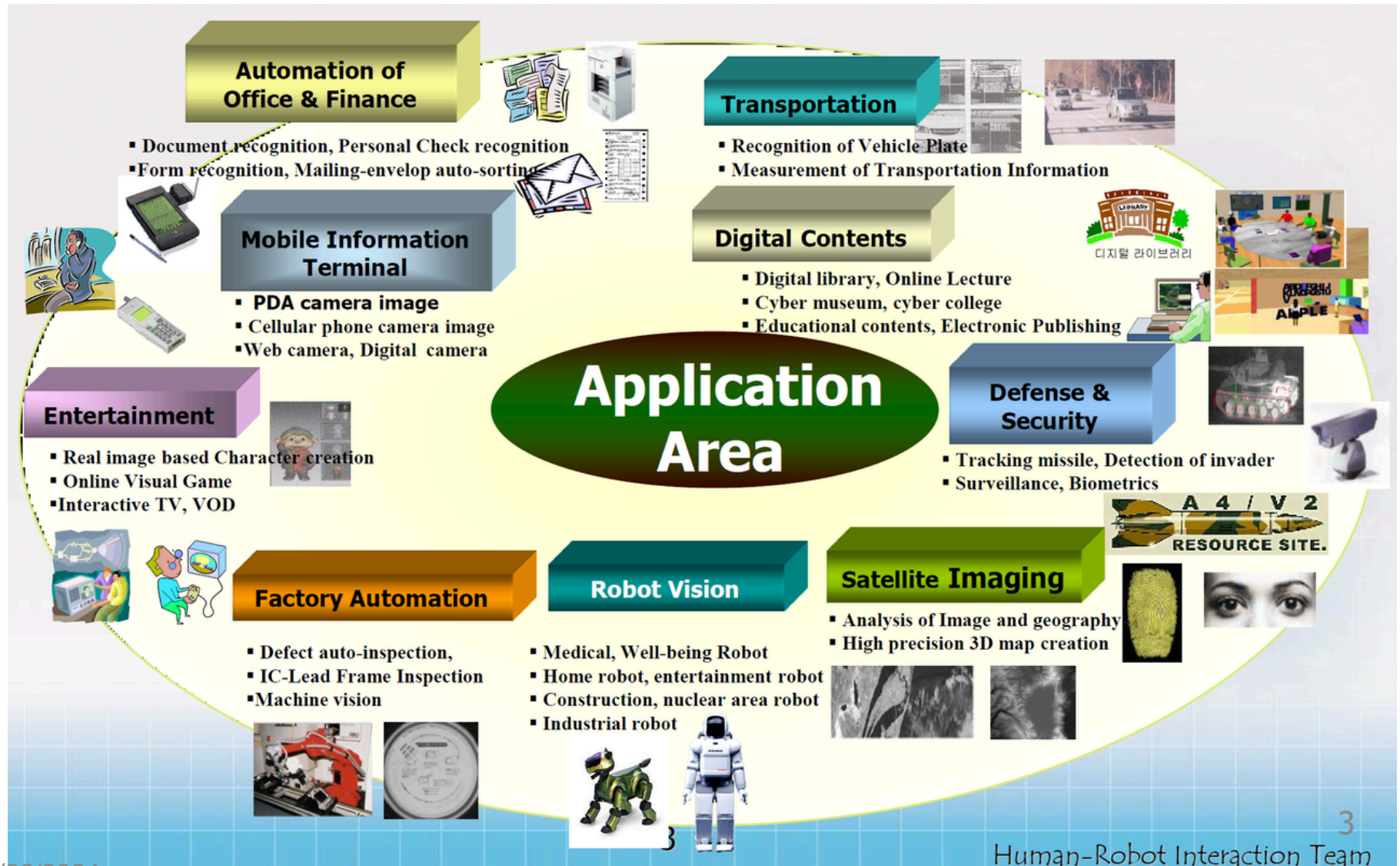
<https://www.python.org/>

Low level Lang. vs **High level Lang.**
General vs Targeted
Compiled vs **Interpreted**

Python is relative **young** but one of the most **popular** programming languages, and **Open software**.

WHAT IS COMPUTATION ?

Computing Application 1: Computer Vision Technology



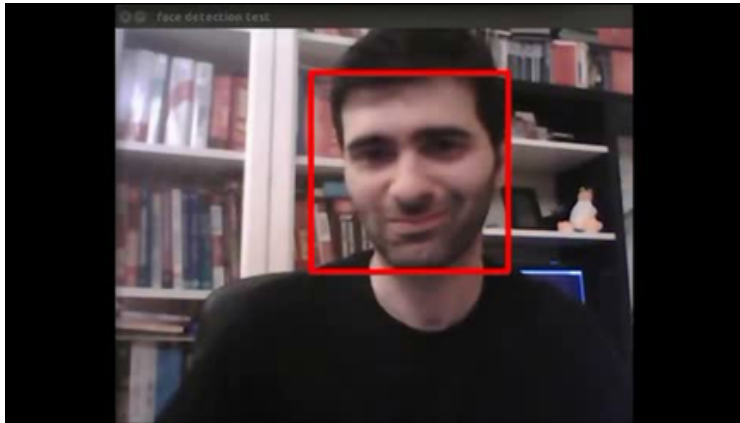
WHAT IS COMPUTATION ?

Computing Application 2: Human Recognition Demo



WHAT IS COMPUTATION ?

Computing Application 3: Face Recognition & Detection Demo



1. Detecting faces using OpenCV



2) Detecting faces using
OpenCV & OpenCL

from movie "Matrix"

WHAT IS COMPUTATION ?

Computer science is not computer programming.

We teach programming to teach computational thinking:

- Solving problems (with a computer)
- Thinking on multiple levels of abstraction
- Decompose a problem into smaller problems
- Thinking about recipes (algorithms)

How to convert it to a program: **coding and debugging**

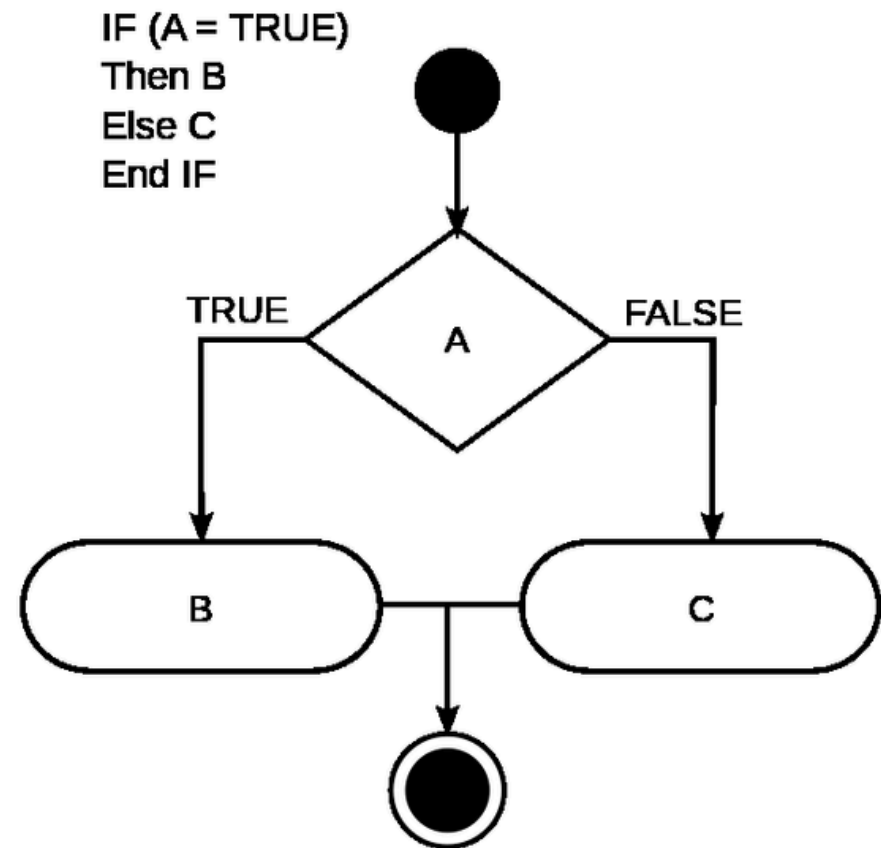
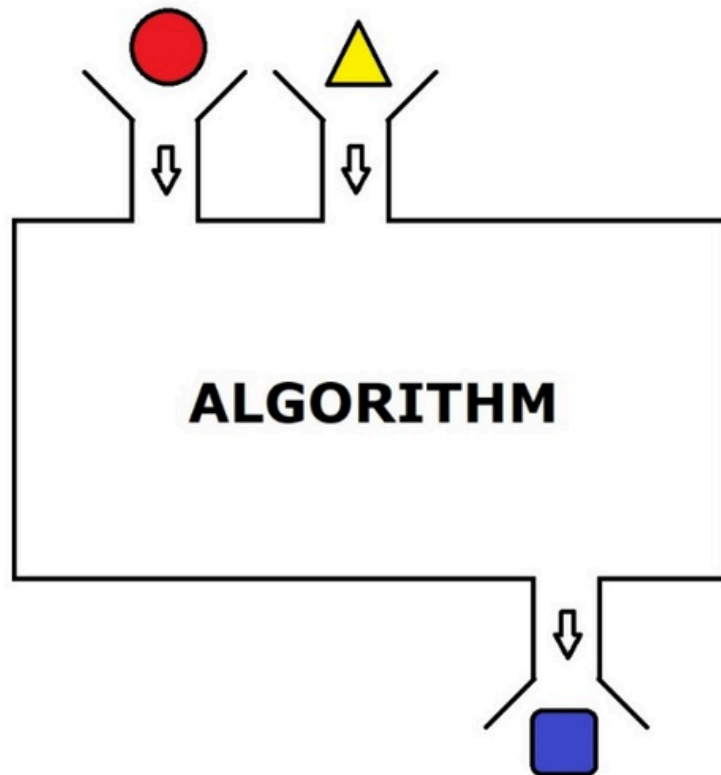
What to do with **computers** ?

How to design an algorithm : **top-down design**

What is a program?

A program is a sequence of instructions that solves some problem(or achieves some effect).

A program implements an algorithm(a recipe for solving a problem).



Coding and debugging

Coding is “a process of fighting with bugs (errors).”

Kinds of errors:

- **Syntax error:** Python cannot understand your program, and refuses to execute it.
- **Runtime error:** At runtime, your program suddenly terminates with an error message.
- **Semantic error:** Your program runs without error messages, but does not do what it is supposed to do.

Why making **such bugs (errors)** ?

- Well, ... , that is the **difference** between **humans** and **computers**.

Why programming ?

Every scientist and engineer must know some programming.

It is part of basic education, like calculus, linear algebra, introductory physics and chemistry, or English.

Alan Perlis 1961

After half a century later, we should change it as follows:

Every student in a university should learn some programming.

It is part of basic education, like calculus, linear algebra, introductory physics and chemistry, or English.

What to do with computers?

According to **Turing-Church Thesis (in 1936)**,
modern computers are essentially equivalent to
a **stored program computer**.

What kind of problems can we solve with a stored
program machine ?

Decidable problems

**Tractable(easy to understand) problems : good
algorithms**

Intractable problems: no good algorithms

e.g., travelling salesman's problem

approximate algorithms

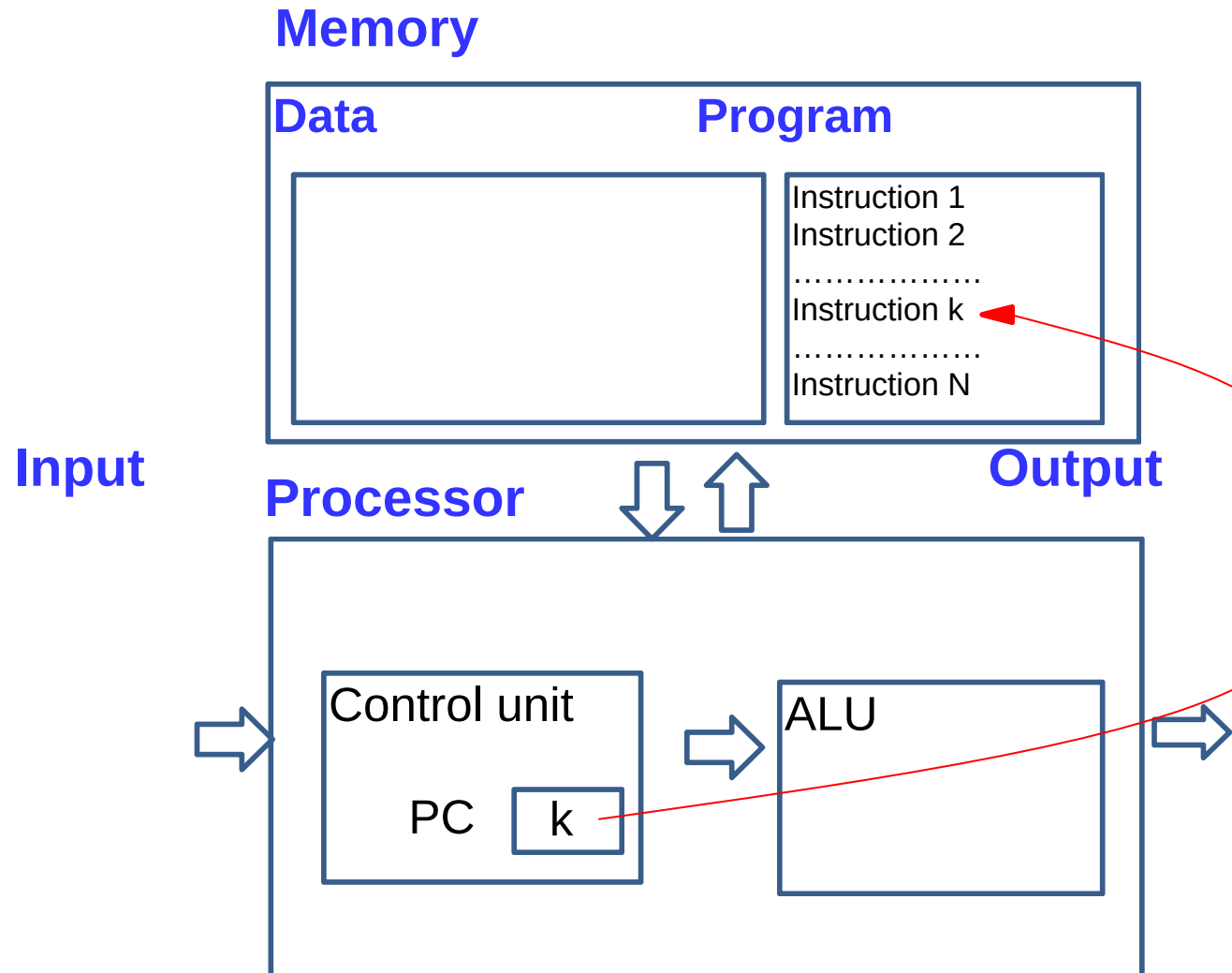
Undecidable problems: no algorithms ever found

e.g. halting problem

- **Atanasoff and Berry(1941): a linear equation solver**
 - The **Atanasoff–Berry computer (ABC)** was the first automatic electronic digital computer,
 - an early electronic digital computing device
- **Alan Turing: bombe machine**

Stored program computers

(* Human brain cells: + 100 billion neurons*)



A small grid-like 2D world

Basic actions

`move ()`: moving one grid forward

`turn_left ()`: turning left by 90

`pick_beeper()`: pick ing up beepers

`drop_beeper()`: putting down beepers

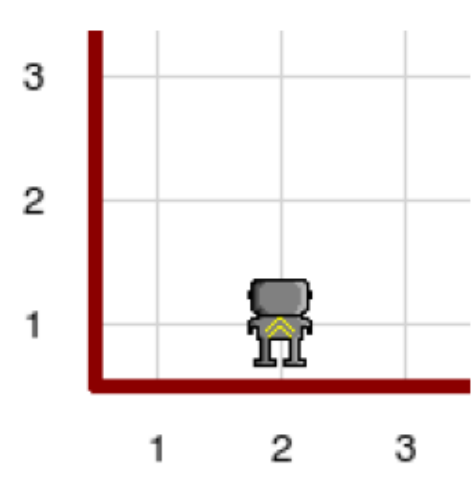
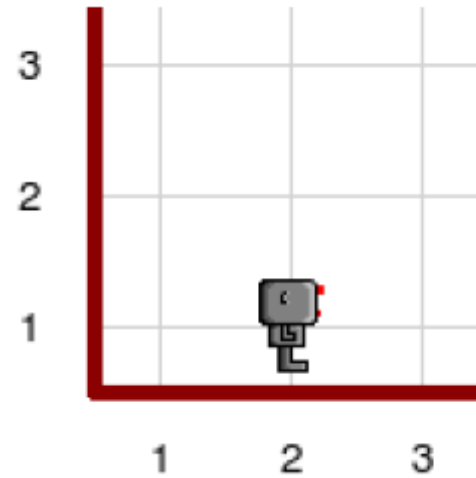
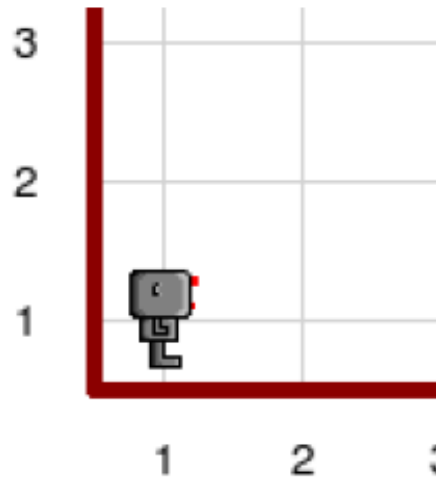
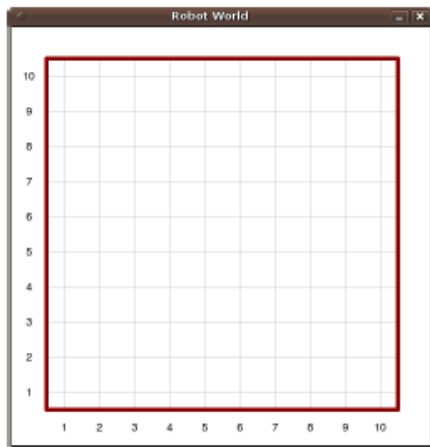
Ourowninstructions:functions

Comments

Interactive mode

Python programs (scripts)

Interactive mode



```
>>>from cs1robots import *  
>>>create_world()  
      >>>hubo = Robot()  
            >>>hubo.move()  
                  >>>hubo.left_turn()
```

Script mode

```
from cs1robots import *  
create_world()  
hubo = Robot()  
hubo.move()  
hubo.turn_left()
```

Functions

A **function definition** specifies the **name** of a function and the **sequence of statements** that are executed when the function is called.

keyword **Colon**

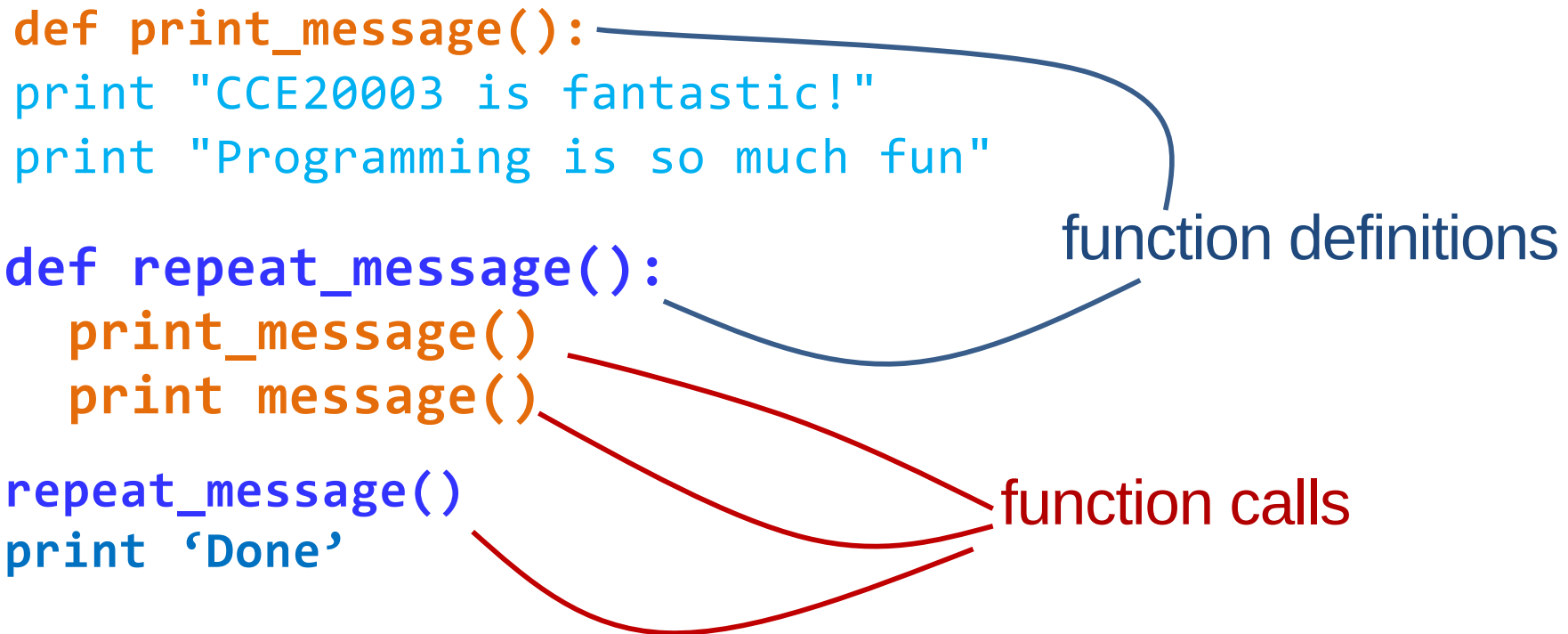
```
def print_message():  
    print "CCE20003 is fantastic!"  
    print "Programming is fun!"
```

identification

You can call a function inside another function:

```
def repeat_message():  
    print_message()  
    print_message()
```

Flow of execution



Execution begins at the first statement. Statements are executed **one by one, top to bottom.**

Function definitions do not change the flow of execution but only define a function.

Function calls are like detours in the flow of **execution**.

Robots

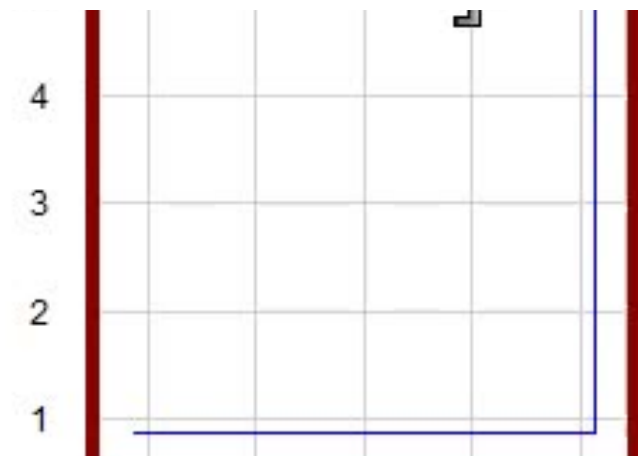
```
# create a robot with one beeper(comments)
hubo = Robot(beepers = 1)
```

object with default parameters

```
# move one step forward
hubo.move()
```

```
# turn left 90 degrees
hubo.turn_left()
```

method:dot notation





Turning right

Define a function!

```
def turn_right():  
    hubo.turn_left()  
    hubo.turn_left()  
    hubo.turn_left()
```

Top-down design

Decomposing a problem into **smaller sub-problems**

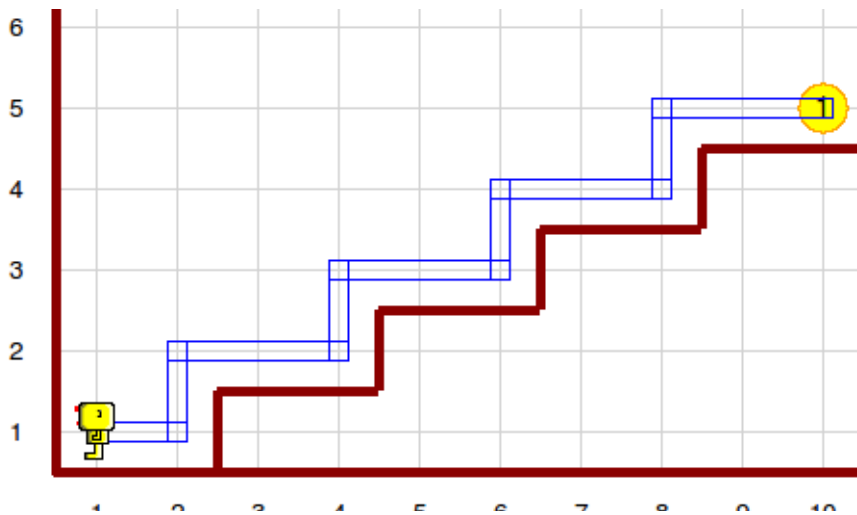
Decompose each of the smaller sub-problems **recursively** until every sub-problem is simple enough to map to a few instructions in a program language

Multi-level abstraction(representing objects at multilevels)

Divide and conquer(breaking prob.in to two or more sub problems)

Newspaper delivery

Hubo should climb the stairs to the front door, drop a newspaper there, and return to his starting point.



Algorithm(pseudo code): Python version:

Climb up four stairs

```
climb_up_four_stairs()
```

Drop the newspaper

```
hubo.drop_beeper()
```

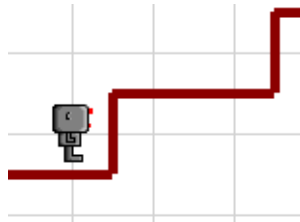
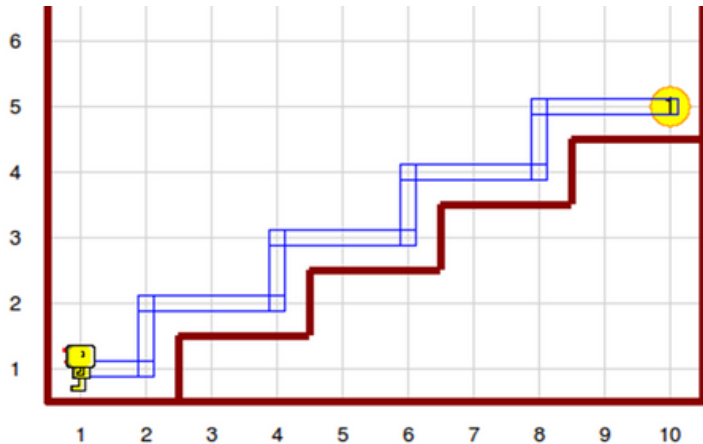
Turn around

```
turn_around()
```

Climb down four stairs

```
climb_down_four_stairs()
```

Climbing up stairs



```
def climb_up_four_stairs():  
    climb_up_one_stair()  
    climb_up_one_stair()  
    climb_up_one_stair()  
    climb_up_one_stair()  
def climb_up_one_stair():  
    hubo.turn_left()  
    hubo.move()  
    turn_right()  
    hubo.move()  
    hubo.move()  
def turn_around():  
    hubo.turn_left()  
    hubo.turn_left()
```

Iteration: for-loops

We should **avoid writing** the same code **repeatedly**.
A for-loop allows us to write it more elegantly:

```
def climb_up_four_stairs():  
    climb_up_one_stair()  
    climb_up_one_stair()  
    climb_up_one_stair()  
    climb_up_one_stair()
```

```
def climb_up_four_stairs():  
    for i in range(4):  
        climb_up_one_stair()
```

To repeat the same instruction 4 times:

```
for i in range(4):
```

← for-loop

```
    print "CCE20003 is fantastic!"
```

Don't forget the indentation!

What is the difference between the following two programs?

```
for i in range(4):
    print "CCE20003 is great!"
    print "I love programming!"
for i in range(4):
    print "CCE20003 is great!"
print "I love programming!"
```

- Practical Programming: An Introduction to Computer Science Using Python 3.0 (3rd Edition) by Paul Gries, Jennifer Campbell, and Jason Montojo, Pragmatic Bookshelf, 2017, ISBN 978-1680502688
- Chapter 1 of the textbook Learning programming with robots (You may download the PDF file on Hisnet site)